

Swiggy Delivery Time , Speed and Price Classification Prediction Project using Traditional Machine Learning Algorithms

In []:

In []:

In [242]:

```
#Import required libraries:
import pandas as pd
import numpy as np
import joblib
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import ExtraTreesRegressor, AdaBoostRegressor
from xgboost import XGBRegressor

from sklearn.model_selection import train_test_split , cross_validate , KFold

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import ExtraTreesClassifier , AdaBoostClassifier
from xgboost import XGBClassifier
from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    r2_score,
    mean_squared_error,
    mean_absolute_error
```

```
)

from sklearn.model_selection import GridSearchCV

import warnings
warnings.filterwarnings("ignore")
```

In []:

In [243]:

```
#Load the dataset:
df = pd.read_csv("dataset/swiggy.csv")
df.head()
```

Out[243]:

	ID	Area	City	Restaurant	Price	Avg ratings	Total ratings	Food type	Address	Delivery time
0	211	Koramangala	Bangalore	Tandoor Hut	300.0	4.4	100	Biryani,Chinese,North Indian,South Indian	5Th Block	59
1	221	Koramangala	Bangalore	Tunday Kababi	300.0	4.1	100	Mughlai,Lucknowi	5Th Block	56
2	246	Jogupalya	Bangalore	Kim Lee	650.0	4.4	100	Chinese	Double Road	50
3	248	Indiranagar	Bangalore	New Punjabi Hotel	250.0	3.9	500	Indian,Punjabi,Tandoor,Chinese	80 Feet Road	57
4	249	Indiranagar	Bangalore	Nh8	350.0	4.0	50	Rajasthani,Gujarati,North Indian,Snacks,Desser...	80 Feet Road	63

In [244]:

```
#Check for any null-values:
df.isnull().sum()
```

Out[244]:

```
ID          0
Area        0
City        0
Restaurant  0
Price       0
Avg ratings 0
Total ratings 0
Food type   0
Address     0
Delivery time 0
dtype: int64
```

In [245]:

```
#Check for any duplicate values:
df.duplicated().sum()
```

Out[245]:

```
np.int64(0)
```

In [246]:

```
df.describe()
```

Out[246]:

```

ID          Price  Avg ratings  Total ratings  Delivery time
```

count	8680.000000	8680.000000	8680.000000	8680.000000	8680.000000
mean	244812.071429	348.444470	3.655104	156.634793	53.967051
std	158671.617188	230.940074	0.647629	391.448014	14.292335
min	211.000000	0.000000	2.000000	20.000000	20.000000
25%	72664.000000	200.000000	2.900000	50.000000	44.000000
50%	283442.000000	300.000000	3.900000	80.000000	53.000000
75%	393425.250000	400.000000	4.200000	100.000000	64.000000
max	466928.000000	2500.000000	5.000000	10000.000000	109.000000

In [247]:

```
#View basic information about the dataset:
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8680 entries, 0 to 8679
Data columns (total 10 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   ID                    8680 non-null   int64
 1   Area                  8680 non-null   object
 2   City                  8680 non-null   object
 3   Restaurant             8680 non-null   object
 4   Price                  8680 non-null   float64
 5   Avg ratings           8680 non-null   float64
 6   Total ratings         8680 non-null   int64
 7   Food type             8680 non-null   object
 8   Address                8680 non-null   object
 9   Delivery time         8680 non-null   int64
dtypes: float64(2), int64(3), object(5)
memory usage: 678.3+ KB
```

In [248]:

```
# Basic EDA:

# Set style
sns.set(style="whitegrid")

# Create 2x3 subplot grid
fig, axes = plt.subplots(2, 3, figsize=(20, 15))

# ===== ROW 1 =====
# 1. Delivery Time Distribution
sns.histplot(df['Delivery time'], kde=True, ax=axes[0, 0])
axes[0, 0].set_title("Delivery Time Distribution")

# 2. Price Distribution
sns.histplot(df['Price'], kde=True, ax=axes[0, 1])
axes[0, 1].set_title("Price Distribution")

# 3. Avg Ratings Distribution
sns.histplot(df['Avg ratings'], kde=True, ax=axes[0, 2])
axes[0, 2].set_title("Average Ratings Distribution")

# ===== ROW 2 =====
# 4. Boxplot of Delivery Time
sns.boxplot(y=df['Delivery time'], ax=axes[1, 0])
axes[1, 0].set_title("Delivery Time Boxplot")

# 5. Top 5 Cities Count
top_cities = df['City'].value_counts().head(5)
sns.barplot(x=top_cities.index, y=top_cities.values, ax=axes[1, 1])
axes[1, 1].set_title("Top 5 Cities")
```

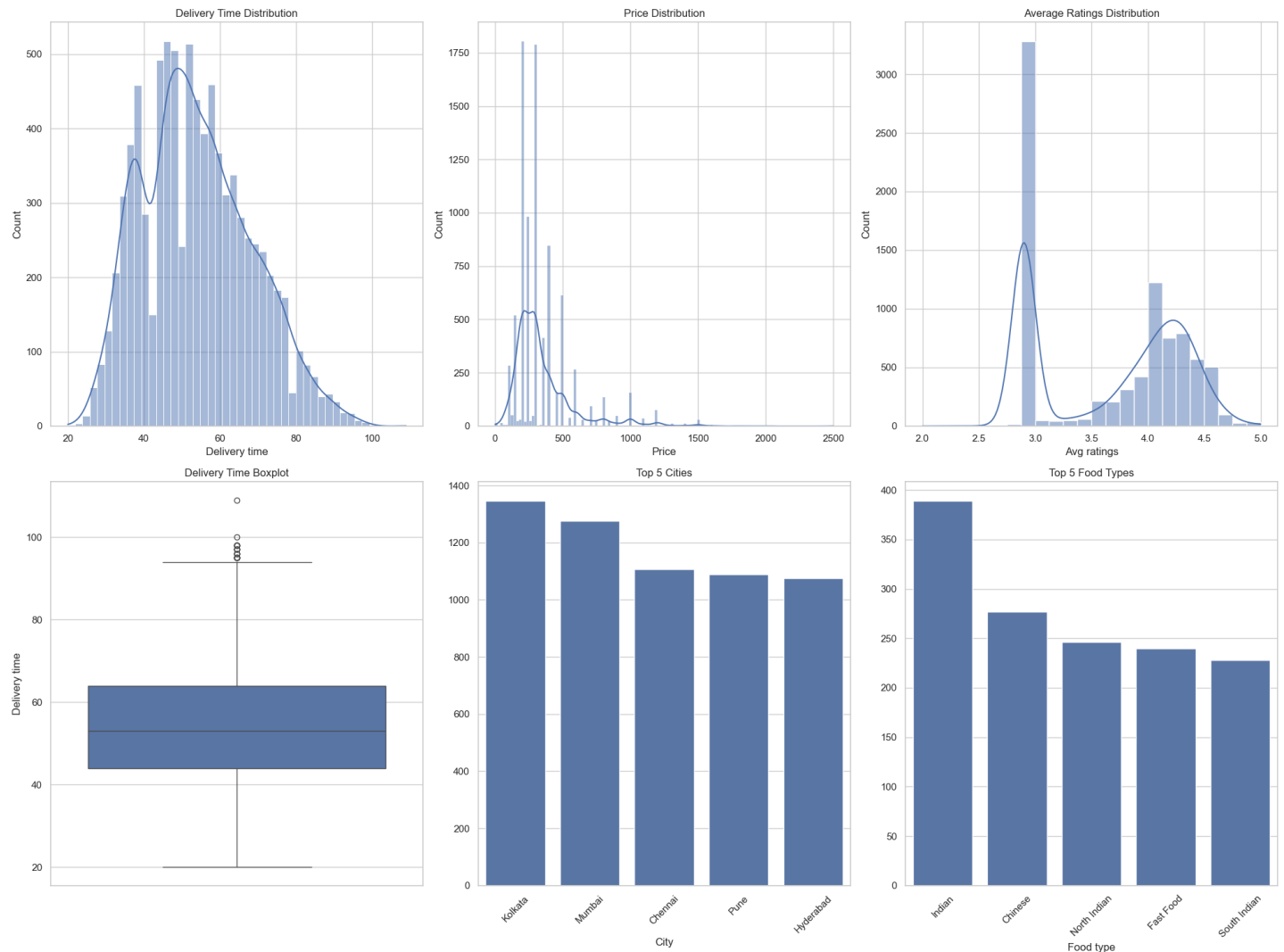
```
axes[1, 1].tick_params(axis='x', rotation=45)
```

```
# 6. Top 5 Food Types Count
```

```
top_food_types = df['Food type'].value_counts().head(5)
sns.barplot(x=top_food_types.index, y=top_food_types.values, ax=axes[1, 2])
axes[1, 2].set_title("Top 5 Food Types")
axes[1, 2].tick_params(axis='x', rotation=45)
```

```
# Adjust layout
```

```
plt.tight_layout()
plt.show()
```



```
In [ ]:
```

```
In [ ]:
```

Q1] Predict the Delivery time of an order using restaurant, pricing, ratings, and location features.

```
In [249]:
```

```
#Feature / Target Split:
```

```
X = df.drop(columns=['Delivery time', 'ID', 'Food type'])
y = df['Delivery time']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
)

#Preprocessing:
num_cols = X_train.select_dtypes(include=['int64', 'float64']).columns
cat_cols = X_train.select_dtypes(include=['object']).columns

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
])
```

In [250]:

```
#Linear Regression:
lr_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', LinearRegression())
])

#KNN Regressor:
knn_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', KNeighborsRegressor())
])

#Decision-tree Regressor:
dt_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', DecisionTreeRegressor(random_state=42))
])

#Extra-trees Regressor:
et_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', ExtraTreesRegressor(n_estimators=250, max_depth=5, random_state=42))
])

#Adaboost:
ada_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', AdaBoostRegressor(
        estimator=DecisionTreeRegressor(max_depth=7),
        n_estimators=400,
        learning_rate=0.5,
        random_state=42
    ))
])

#XG-Boost:
xgb_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', XGBRegressor(
        n_estimators=500,
        learning_rate=0.05,
        max_depth=6,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
        n_jobs=-1
    ))
])
```

In [251]:

```
#Model dictionary:
models = {
```

```

"Linear Regression": lr_pipeline,
"KNN": knn_pipeline,
"Decision Tree": dt_pipeline,
"Extra Trees": et_pipeline,
"AdaBoost": ada_pipeline,
"XGBoost": xgb_pipeline
}

```

In [252]:

```

#Cross validation:

kf = KFold(n_splits=5, shuffle=True, random_state=42)

scoring = {
    'r2': 'r2',
    'mae': 'neg_mean_absolute_error',
    'rmse': 'neg_root_mean_squared_error'
}

results = []

for name, model in models.items():

    cv_results = cross_validate(
        model,
        X_train,
        y_train,
        cv=kf,
        scoring=scoring
    )

    mae = -np.mean(cv_results['test_mae'])
    rmse = -np.mean(cv_results['test_rmse'])
    r2 = np.mean(cv_results['test_r2'])

    results.append({
        "Model": name,
        "CV R2": round(r2, 4),
        "CV MAE": round(mae, 2),
        "CV RMSE": round(rmse, 2)
    })

results_df = pd.DataFrame(results).sort_values(by="CV R2", ascending=False)
print(results_df)

```

	Model	CV R2	CV MAE	CV RMSE
5	XGBoost	0.6644	6.33	8.25
1	KNN	0.6505	6.27	8.42
2	Decision Tree	0.5216	7.22	9.85
0	Linear Regression	0.4763	7.69	10.26
3	Extra Trees	0.3527	9.18	11.46
4	AdaBoost	0.3438	9.47	11.54

In [253]:

```

#Train the best model on Full-training data:
best_model = results_df.iloc[0]['Model']
print("Best Model:", best_model)

best_pipeline = models[best_model]
best_pipeline.fit(X_train, y_train)

```

Best Model: XGBoost

Out[253]:

```

► Pipeline
    i ?
    ► preprocessing:
      ColumnTransformer

```

?

- ▶ num
- ▶ StandardScaler
- ▶ cat
- ▶ OneHotEncoder
- ▶ XGBRegressor

?

?

?

In [254]:

```
#Test Evaluation on testing:
```

```
y_pred = best_pipeline.predict(X_test)

print("TEST RESULTS")
print("R2:", r2_score(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
```

```
TEST RESULTS
R2: 0.675024151802063
MAE: 6.412059307098389
RMSE: 8.241580250142036
```

In [255]:

```
#Perform Hyperparamter tuning on the best pipeline:
from sklearn.model_selection import GridSearchCV
```

```
param_grid = {
    'model__n_estimators': [100, 200, 300],
    'model__max_depth': [3, 5, 7],
    'model__learning_rate': [0.01, 0.05, 0.1],
    'model__subsample': [0.8, 1.0],
    'model__colsample_bytree': [0.8, 1.0]
}
```

```
grid_search = GridSearchCV(
    estimator=best_pipeline,
    param_grid=param_grid,
    scoring='neg_root_mean_squared_error',
    cv=5,
    n_jobs=-1,
    verbose=2
)
```

```
grid_search.fit(X_train, y_train)
```

```
rmse = mean_squared_error(y_test, y_pred) ** 0.5
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
print(f"\nRMSE: {rmse:.4f}")
print(f"MAE : {mae:.4f}")
print(f"R2 : {r2:.4f}")
```

Fitting 5 folds for each of 108 candidates, totalling 540 fits

```
RMSE: 8.2416
MAE : 6.4121
R2 : 0.6750
```

In [256]:

```
#Now test on one single tuple:
```

```
single_data = pd.DataFrame({
    'Area': ['Vastrapur'],
    'City': ['Ahmedabad'],
    'Restaurant': ['Vanilla Sky'],
    'Price': [300.0],
    'Avg ratings': [2.9],
    'Address': ['Vastrapur'],
    'Total ratings': [220]
})

pred = grid_search.predict(single_data)
print("Predicted Delivery Time:", round(pred[0]), "minutes")
print("Best Model Name : ", grid_search)
```

Predicted Delivery Time: 48 minutes

```
Best Model Name : GridSearchCV(cv=5,
    estimator=Pipeline(steps=[('preprocessing',
                                ColumnTransformer(transformers=[('num',
                                                                    StandardScaler
                                                                    (
                                                                    ),
                                                                    Index(['Price'
                                                                    , 'Avg ratings', 'Total ratings'], dtype='object')),
                                                                    ('cat',
                                                                    OneHotEncoder(
                                                                    handle_unknown='ignore'),
                                                                    Index(['Area',
                                                                    'City', 'Restaurant', 'Address'], dtype='object')))])),
                                ('model',
                                XGBRegressor(base_score=None,
                                                booster=None,
                                                callbacks=...
                                                missing=nan,
                                                monotone_constraints=None,
                                                multi_strategy=None,
                                                n_estimators=500,
                                                n_jobs=-1,
                                                num_parallel_tree=None, ...))),
                                n_jobs=-1,
                                param_grid={'model__colsample_bytree': [0.8, 1.0],
                                                'model__learning_rate': [0.01, 0.05, 0.1],
                                                'model__max_depth': [3, 5, 7],
                                                'model__n_estimators': [100, 200, 300],
                                                'model__subsample': [0.8, 1.0]},
                                scoring='neg_root_mean_squared_error', verbose=2)
```

In []:

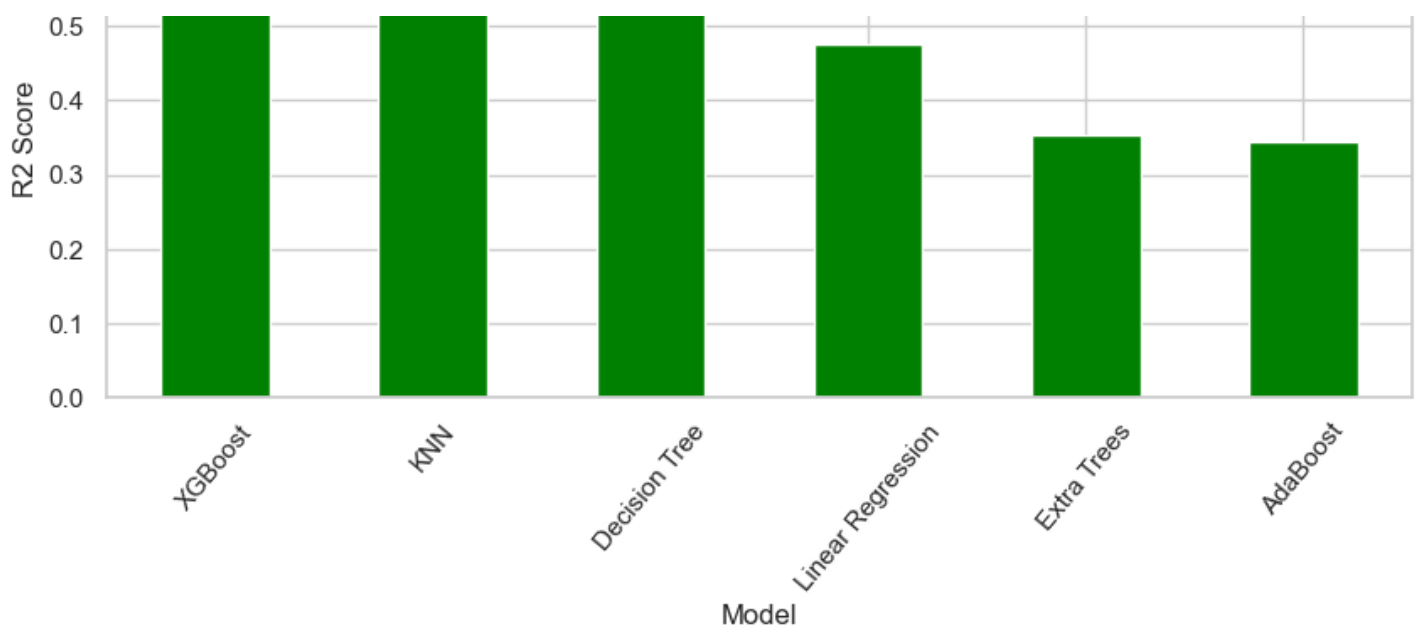
In [257]:

```
#Model comparison:
```

```
plt.figure(figsize=(10,4))
results_df.set_index("Model")["CV R2"].plot(kind='bar',color="green")
plt.title("Model Comparison (R2 Score)")

plt.xlabel("Model")
plt.ylabel("R2 Score")
plt.xticks(rotation=50)
plt.show()
```





In []:

In [258]:

```
#Export the best-performing model:  
joblib.dump(grid_search, "Swiggy_FoodDeliveryTimePrediction_Model.pkl")
```

Out[258]:

```
['Swiggy_FoodDeliveryTimePrediction_Model.pkl']
```

In []:

CONCLUSION : We used several traditional-ml algorithms to predict the delivery time. The predicted delivery time is 48 minutes.

In []:

In []:

In []:

In []:

Q2] Can we classify whether a restaurant will have fast or slow delivery based on its price, ratings, and popularity?

In [259]:

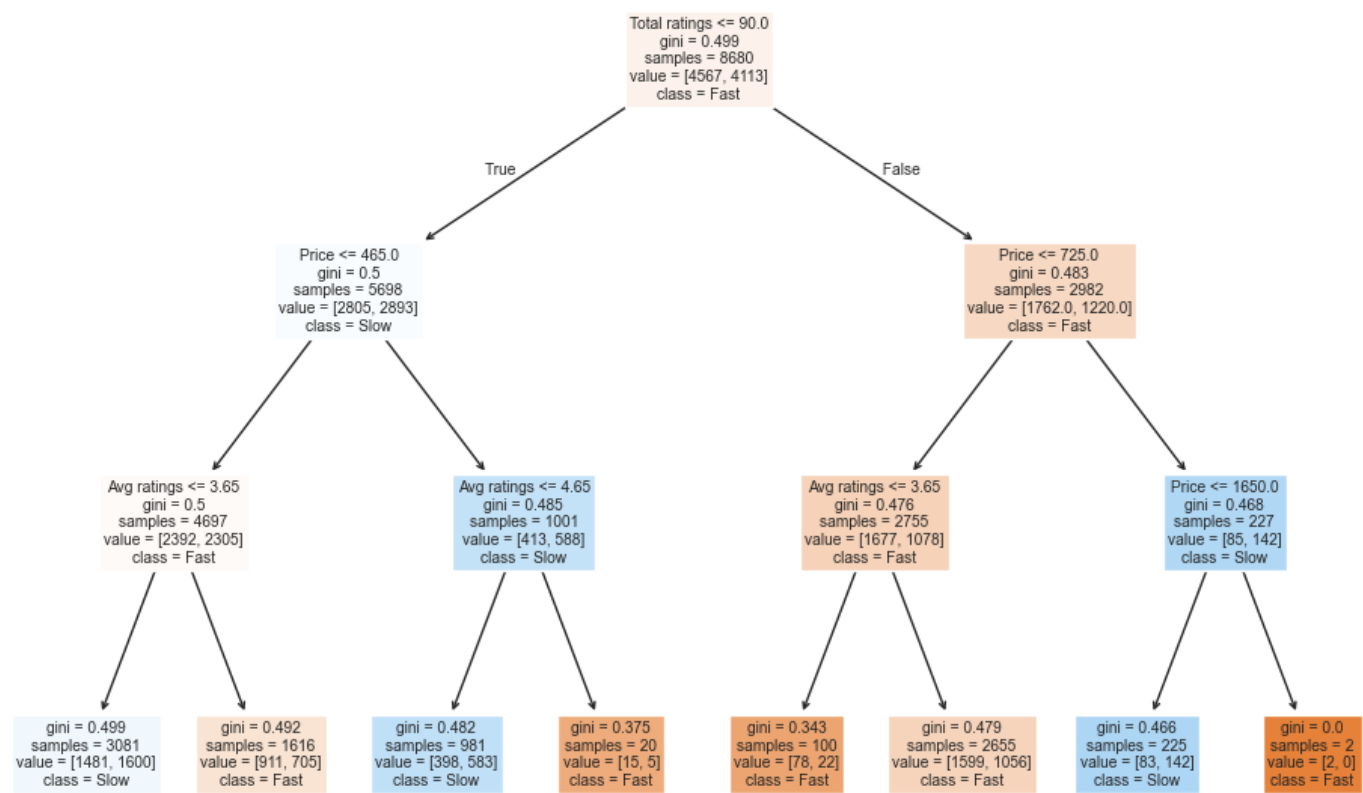
```
df['delivery_speed'] = df['Delivery time'].apply(  
    lambda x: 'Fast' if x <= df['Delivery time'].median() else 'Slow'  
)
```

```
X = df[['Price', 'Avg ratings', 'Total ratings']]
y = df['delivery_speed']

model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X, y)

plt.figure(figsize=(12,8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=['Fast', 'Slow'],
    filled=True
)
plt.title("Decision Tree for Delivery Speed Classification")
plt.show()
```

Decision Tree for Delivery Speed Classification



CONCLUSION]

- Expensive Restaurants tend to deliver food faster.
- Restaurants having a more preference (based on average user ratings) tend to deliver food faster.
- Restaurants having low ratings (based on average user ratings) tend to deliver food slower.

In [260]:

```
df.head()
```

Out[260]:

ID	Area	City	Restaurant	Price	Avg ratings	Total ratings	Food type	Address	Delivery time	delive
----	------	------	------------	-------	-------------	---------------	-----------	---------	---------------	--------

0	219	Koramangala	Bangalore	City Restaurant	Tandoor Hut	Price	Avg ratings	Total ratings	Biryani,Chinese, North Indian, South Indian	5Th Block	Delivery time	delivery speed
1	221	Koramangala	Bangalore		Tunday Kababi	300.0	4.1	100	Mughlai,Lucknowi	5Th Block	56	
2	246	Jogupalya	Bangalore		Kim Lee	650.0	4.4	100	Chinese	Double Road	50	
3	248	Indiranagar	Bangalore		New Punjabi Hotel	250.0	3.9	500	Indian,Punjabi,Tandoor,Chinese	80 Feet Road	57	
4	249	Indiranagar	Bangalore		Nh8	350.0	4.0	50	Rajasthani,Gujarati,North Indian,Snacks,Desser...	80 Feet Road	63	



In []:

Q3] Predict whether a restaurant belongs to Premium or Budget segment using features like Avg Ratings, Total Ratings, Food Type, and Delivery Time (Binary Classification).

In [261]:

```
df['price_segment'] = df['Price'].apply(lambda x: 'Premium' if x > 500 else 'Budget')

print("Check for imbalance in 'price_segment' attribute:")
print(df.price_segment.value_counts())
print("\n\n\n\n\n\n\n")

X = df.drop(columns=['price_segment', 'Price', 'ID', 'Delivery time', 'delivery_speed'])
y = df['price_segment']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

num_cols = X_train.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols = X_train.select_dtypes(include=['object']).columns.tolist()

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=True), cat_cols)
])

le = LabelEncoder()
y_train_encoded = le.fit_transform(y_train)
y_test_encoded = le.transform(y_test)

# All Classification Pipelines:
lr_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', LogisticRegression(max_iter=1000, random_state=42))
])

knn_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', KNeighborsClassifier(n_neighbors=7, n_jobs=-1))
])

et_pipeline = Pipeline([
```

```

        ('preprocessing', preprocessor),
        ('model', ExtraTreesClassifier(n_estimators=120, max_depth=9, n_jobs=-1, random_state
=42))
    ])

ada_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=4),
        n_estimators=200,
        learning_rate=0.8,
        random_state=42
    ))
])

xgb_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', XGBClassifier(
        n_estimators=300,
        learning_rate=0.1,
        max_depth=6,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
        n_jobs=-1,
        tree_method='hist',
        eval_metric='logloss'
    ))
])

# Cross Validation:
## Model Dictionary:

models = {
    "Logistic Regression": lr_pipeline,
    "KNN Classifier": knn_pipeline,
    "Extra Trees": et_pipeline,
    "AdaBoost": ada_pipeline,
    "XGBoost": xgb_pipeline
}

kf = KFold(n_splits=3, shuffle=True, random_state=42)    # 3 folds = faster
scoring = ['accuracy', 'f1_weighted']

results = []

for name, model in models.items():
    print(f"□ Running {name}...")    # Progress tracking
    if name == "XGBoost":
        cv_results = cross_validate(model, X_train, y_train_encoded,
                                    cv=kf, scoring=scoring, n_jobs=-1)
    else:
        cv_results = cross_validate(model, X_train, y_train,
                                    cv=kf, scoring=scoring, n_jobs=-1)

    results.append({
        "Model": name,
        "CV Accuracy": round(np.mean(cv_results['test_accuracy']), 4),
        "CV F1 Score": round(np.mean(cv_results['test_f1_weighted']), 4)
    })

results_df = pd.DataFrame(results).sort_values(by="CV Accuracy", ascending=False)
print("\n" + "="*50)
print(results_df)

```

Check for imbalance in 'price_segment' attribute:
price_segment

```
Budget      7665
Premium     1015
Name: count, dtype: int64
```

```
❑ Running Logistic Regression...
❑ Running KNN Classifier...
❑ Running Extra Trees...
❑ Running AdaBoost...
❑ Running XGBoost...
```

```
=====
      Model  CV Accuracy  CV F1 Score
0  Logistic Regression    0.8895    0.8480
3          AdaBoost      0.8871    0.8405
2      Extra Trees      0.8831    0.8282
4          XGBoost      0.8823    0.8472
1      KNN Classifier    0.8774    0.8434
```

In [262]:

```
#Perform Hyperparamter tuning on the best pipeline:

param_grid = {
    'model__C': [0.001, 0.01, 0.1, 1, 10, 100],
    'model__penalty': ['l1', 'l2'],
    'model__solver': ['liblinear']
}

grid_search = GridSearchCV(
    estimator=lr_pipeline,
    param_grid=param_grid,
    scoring='accuracy',
    cv=5,
    n_jobs=-1,
    verbose=2
)

grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_

y_pred = best_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, pos_label='Premium'))
print("Recall:", recall_score(y_test, y_pred, pos_label='Premium'))
print("F1 Score:", f1_score(y_test, y_pred, pos_label='Premium'))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
Accuracy: 0.8894009216589862
Precision: 0.6122448979591837
Recall: 0.1477832512315271
F1 Score: 0.23809523809523808
```

```
Confusion Matrix:
[[1514   19]
 [ 173   30]]
```

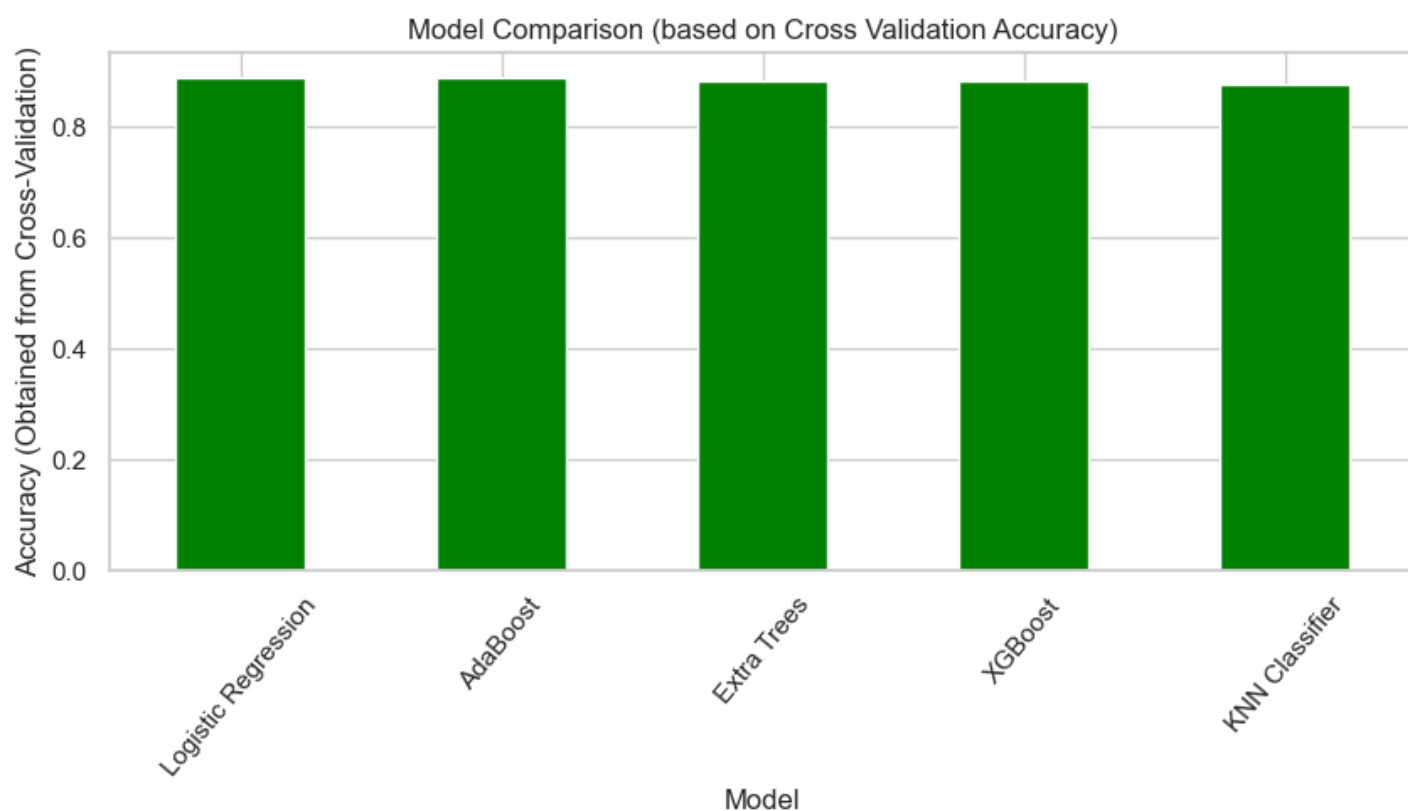
Classification Report:

	precision	recall	f1-score	support
Budget	0.90	0.99	0.94	1533
Premium	0.61	0.15	0.24	203
accuracy			0.89	1736
macro avg	0.75	0.57	0.59	1736
weighted avg	0.86	0.89	0.86	1736

In [263]:

```
#Model comparison:
plt.figure(figsize=(10,4))
results_df.set_index("Model")["CV Accuracy"].plot(kind='bar',color="green")
plt.title("Model Comparison (based on Cross Validation Accuracy)")

plt.xlabel("Model")
plt.ylabel("Accuracy (Obtained from Cross-Validation)")
plt.xticks(rotation=50)
plt.show()
```



In [264]:

```
#Now test on one single tuple:
# ===== Test on One Single Tuple =====
single_data = pd.DataFrame({
    'Area': ['Ashok Nagar'],
    'City': ['Bangalore'],
    'Restaurant': ['Roomali'],
    'Avg ratings': [4.4],
    'Total ratings': [100],
    'Food type': ['North Indian,Indian'],
    'Address': ['Church Street']
})

# Make prediction using the best model
pred = grid_search.predict(single_data)
print("Predicted Price Segment:", pred[0])
print("Best Model Name : ",grid_search)
```

Predicted Price Segment: Premium
Best Model Name : GridSearchCV/Extra

```
Best Model Name : GridSearchCV(cv=5,
                                estimator=Pipeline(steps=[('preprocessing',
                                                            ColumnTransformer(transformers=[('num',
                                                                                               StandardScaler
                                                                                               (
                                                                                               ),
                                                                                               [
                                                                                               ('Avg ',
                                                                                               'ratings',
                                                                                               'Total ',
                                                                                               'ratings'])),
                                                                                               ('cat',
                                                                                               OneHotEncoder(
                                                                                               handle_unknown='ignore'),
                                                                                               [
                                                                                               ('Area',
                                                                                               'City',
                                                                                               'Restaurant',
                                                                                               'Food ',
                                                                                               'type',
                                                                                               'Address'])])
                                                                                               ),
                                                                                               (
                                                                                               ('model',
                                                                                               LogisticRegression(max_iter=1000,
                                                                                               random_state=42)))
                                                                                               ),
                                                                                               n_jobs=-1,
                                                                                               param_grid={'model__C': [0.001, 0.01, 0.1, 1, 10, 100],
                                                                                               'model__penalty': ['l1', 'l2'],
                                                                                               'model__solver': ['liblinear']},
                                                                                               scoring='accuracy', verbose=2)
                                                                                               ),
                                                                                               (
                                                                                               ('model',
                                                                                               LogisticRegression(max_iter=1000,
                                                                                               random_state=42)))
                                                                                               ),
                                                                                               n_jobs=-1,
                                                                                               param_grid={'model__C': [0.001, 0.01, 0.1, 1, 10, 100],
                                                                                               'model__penalty': ['l1', 'l2'],
                                                                                               'model__solver': ['liblinear']},
                                                                                               scoring='accuracy', verbose=2)
```

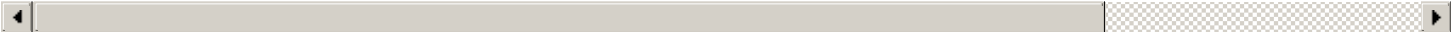
In [265]:

```
#Observe the specifications in the 2nd record:
df.iloc[50:]
```

Out[265]:

	ID	Area	City	Restaurant	Price	Avg ratings	Total ratings	Food type	Address	Delivery time
50	2839	Ashok Nagar	Bangalore	C Sharp Kitchen	300.0	2.9	80	Chinese,Continental,Fast Food	Church Street	33
51	2840	Ashok Nagar	Bangalore	Roomali	1000.0	4.4	100	North Indian,Indian	Church Street	35
52	2843	Ashok Nagar	Bangalore	Tadka Singh	300.0	4.2	100	Punjabi,North Indian	Church Street	38
53	2914	Ashok Nagar	Bangalore	Bheemas	550.0	4.3	100	Andhra	Church Street	31
54	2915	Vasanth Nagar	Bangalore	Watson'S	450.0	2.9	80	Fast Food	1St Main Road	53
...	...	...	...	...	...	...	...	...	...	...
8675	464626	Panjarapole Cross Road	Ahmedabad	Malt Pizza	500.0	2.9	80	Pizzas	Navrangpura	40
8676	465835	Rohini	Delhi	Jay Mata Ji Home Kitchen	200.0	2.9	80	South Indian	Rohini	28
8677	465872	Rohini	Delhi	Chinese Kitchen King	150.0	2.9	80	Chinese,Snacks,Tandoor	Rohini	58
8678	465990	Rohini	Delhi	Shree Ram Paratha Wala	150.0	2.9	80	North Indian,Indian,Snacks	Rohini	28
8679	466488	Navrangpura	Ahmedabad	Sassy Street	250.0	2.9	80	Chaat,Snacks,Chinese	Navrangpura	44

8630 rows x 12 columns



```
In [266]:
```

```
#Export the best-performing model:
```

```
joblib.dump(grid_search, "Price_Classification.pkl")
```

```
Out[266]:
```

```
['Price_Classification.pkl']
```

```
In [267]:
```

```
#Export both the models to the S3-bucket on AWS:
```

```
import boto3
import os
```

```
# Upload model to S3
```

```
best_model_name = "Price_Classification.pkl"
```

```
bucket_name = "swiggydeliverytimeandpriceclassificationmodels"
```

```
s3_folder = "models_deployed"
```

```
s3_key = s3_folder + "/" + best_model_name
```

```
#Create our S3 Client:
```

```
# Create S3 client
```

```
s3 = boto3.client(
    "s3",
    aws_access_key_id=os.getenv("AWS_ACCESS_KEY_ID"),
    aws_secret_access_key=os.getenv("AWS_SECRET_ACCESS_KEY"),
    region_name=os.getenv("AWS_REGION")
)
```

```
s3.upload_file(
    Filename=best_model_name,
    Bucket=bucket_name,
    Key=s3_key
)
```

```
print("Model uploaded successfully to S3!")
print(f"S3 Path: s3://{bucket_name}/{s3_key}")
```

```
# Upload model to S3
```

```
best_model_name = "Swiggy_FoodDeliveryTimePrediction_Model.pkl"
```

```
bucket_name = "swiggydeliverytimeandpriceclassificationmodels"
```

```
s3_folder = "models_deployed"
```

```
s3_key = s3_folder + "/" + best_model_name
```

```
#Create our S3 Client:
```

```
# Create S3 client
```

```
s3 = boto3.client(
    "s3",
    aws_access_key_id="AKIAZVH6SCDIYHDAXHVF",
    aws_secret_access_key="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",
    region_name="ap-south-1"    # Mumbai region, change if needed
)
```



```
s3.upload_file(  
    Filename=best_model_name,  
    Bucket=bucket_name,  
    Key=s3_key  
)  
  
print("Model uploaded successfully to S3!")  
print(f"S3 Path: s3://{bucket_name}/{s3_key}")
```

Model uploaded successfully to S3!

S3 Path: s3://swiggydeliverytimeandpriceclassificationmodels/models_deployed/Price_Classification.pkl

Model uploaded successfully to S3!

S3 Path: s3://swiggydeliverytimeandpriceclassificationmodels/models_deployed/Swiggy_FoodDeliveryTimePrediction_Model.pkl

PROJECT BY:

- **Name:** Om Satyawan Pathak
- **Email:** omsatyawanpathakgit@gmail.com
- **Linkedin:** www.linkedin.com/in/om-satyawan-pathak-029b02368

In []: